PYTHON CASE STUDIES 91–100

Complete Assignment with Clear Flowcharts

Each case includes: problem statement, objectives, assumptions, algorithm, a readable flowchart, pseudocode, Python source code, test cases, sample output, and conclusion.

Flowchart key: rounded green boxes indicate Start/End; blue boxes indicate processing or decision steps. Arrows show the direction of control flow.

Prepared for academic practice using fundamental Python concepts.

91. Advanced String-Based Registration Record Analyzer

1. Problem Statement

Create a program that reads registration records stored as delimited strings, validates required field lengths and formats, extracts components, detects duplicate registration IDs, and classifies errors.

2. Objectives

• Split structured strings into fields and validate each field.

• Check ID, name, email-like format, and phone length using basic string operations.

• Detect duplicate IDs and report valid records and error categories.

3. Assumptions

• Record format is ID|NAME|EMAIL|PHONE.

• ID must be 5 characters; name must contain at least 2 characters; email must contain '@' and a dot after it; phone must contain 10 digits.

• An ID is considered duplicate if it has already appeared, even if the later record has other errors.

4. Algorithm

1. Read the registration strings.

2. Create an empty list of seen IDs and result counters.

3. For each record, split on '|'.

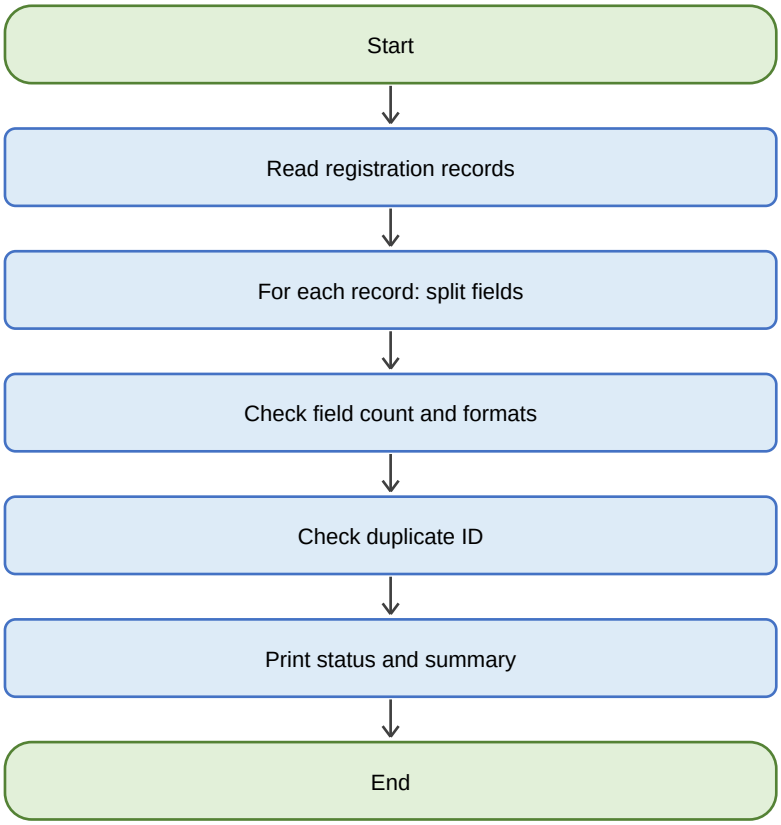
4. Check field count, then validate each field and collect error labels.

5. Check whether the ID is duplicated; add a new non-empty ID to the seen list.

6. Print extracted fields, status, and error labels; summarize valid and invalid records.

5. Flowchart

Follow the arrows from top to bottom. Conditions are stated inside the relevant step boxes.



6. Pseudocode

```
BEGIN
  records ← input list
  seen_ids ← empty list
  FOR each record IN records
    fields ← split record by '|'
    errors ← empty list
    IF field count ≠ 4: add FIELD_COUNT error
    ELSE validate ID, name, email, phone
         IF ID already in seen_ids: add DUPLICATE_ID
         ELSE add non-empty ID to seen_ids
    IF errors empty: print VALID; ELSE print INVALID and errors
  print totals
END
```

7. Python Source Code

```
records = [
    "A1001|Ravi Kumar|ravi@mail.com|9876543210",
    "A1002|Meena|meena.example.com|9876543211",
    "A1001|Arun Rao|arun@mail.com|9876543212",
    "A1003|Jo|jo@site.in|12345",
    "bad-record"
]

seen_ids = []
valid_count = 0
invalid_count = 0

for record in records:
    fields = record.split("|")
    errors = []

    if len(fields) != 4:
        errors.append("FIELD_COUNT")
        reg_id = name = email = phone = ""
    else:
        reg_id, name, email, phone = fields
        if len(reg_id) != 5:
            errors.append("ID_LENGTH")
        if len(name.strip()) < 2:
            errors.append("NAME_LENGTH")
        if "@" not in email or "." not in email.split("@")[-1]:
```

```
            errors.append("EMAIL_FORMAT")
        if len(phone) != 10 or not phone.isdigit():
            errors.append("PHONE_FORMAT")
        if reg_id in seen_ids:
            errors.append("DUPLICATE_ID")
        elif reg_id:
            seen_ids.append(reg_id)

    if errors:
        invalid_count += 1
        print(record, "-> INVALID:", ", ".join(errors))
    else:
        valid_count += 1
        print("VALID:", reg_id, name, email, phone)

print("Valid:", valid_count, "Invalid:", invalid_count)
```

8. Test Cases

Test	Input / Condition	Expected Result
Valid record	A1001|Ravi Kumar|ravi@mail.com|9876543210	VALID
Bad email	A1002|Meena|meena.example.com|9876543211	EMAIL_FORMAT
Repeated ID	A1001|Arun Rao|arun@mail.com|9876543212	DUPLICATE_ID
Wrong field count	bad-record	FIELD_COUNT

9. Sample Output

```
VALID: A1001 Ravi Kumar ravi@mail.com 9876543210
A1002|Meena|meena.example.com|9876543211 -> INVALID: EMAIL_FORMAT
A1001|Arun Rao|arun@mail.com|9876543212 -> INVALID: DUPLICATE_ID
A1003|Jo|jo@site.in|12345 -> INVALID: PHONE_FORMAT
bad-record -> INVALID: FIELD_COUNT
Valid: 1 Invalid: 4
```

10. Conclusion

The program demonstrates string splitting, field validation, duplicate detection, error classification, and summary counting.

92. Multi-Event Sports Scheduling and Scoring

1. Problem Statement

Store sports fixtures and team scores using lists and tuples, identify teams scheduled in overlapping time slots, total team points, and print a final ranking.

2. Objectives

• Represent fixtures as tuples inside a list.

• Detect a team appearing in two events at the same time.

• Aggregate points and sort teams by total points.

3. Assumptions

• Each fixture is (event, time_slot, team1, team2).

• Each score record is (team, points).

• A team earns the points explicitly listed; ties share the same total but are displayed alphabetically.

4. Algorithm

1. Store fixtures and score tuples.

2. Compare every pair of fixtures sharing a time slot.

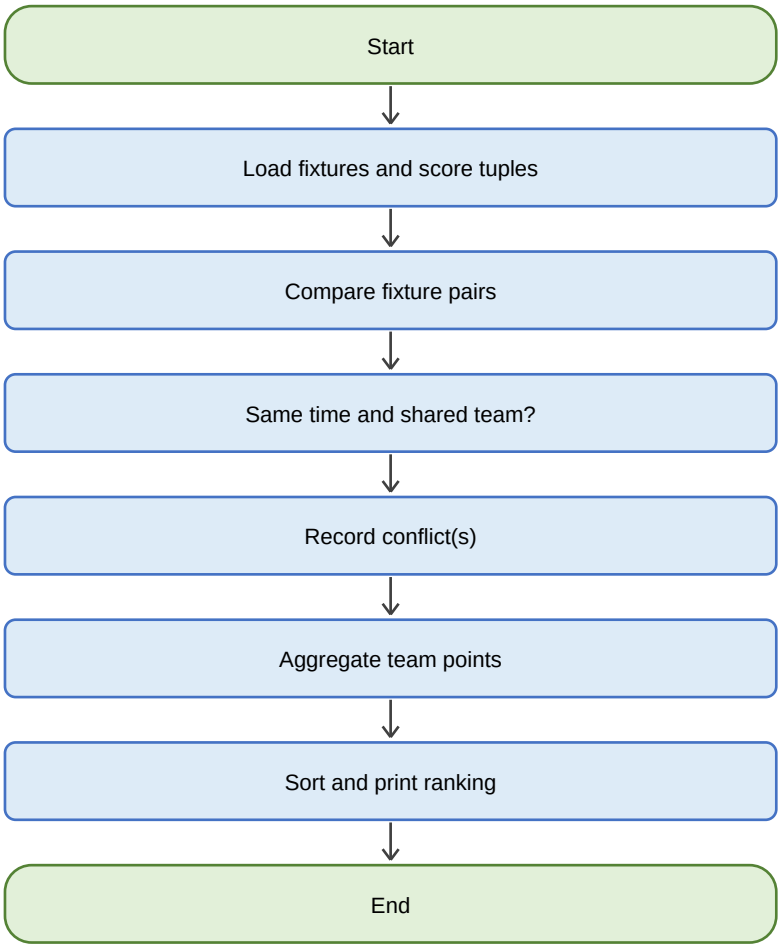
3. If the same team appears in both, report a conflict.

4. Add each score to a team-total list or update its existing total.

5. Sort totals by points descending and team name ascending; display results.

5. Flowchart

Follow the arrows from top to bottom. Conditions are stated inside the relevant step boxes.



6. Pseudocode

```
BEGIN
  fixtures ← list of fixture tuples
  scores ← list of (team, points) tuples
  FOR each pair of fixtures
    IF time slots equal AND a team is shared: report conflict
  totals ← empty list
  FOR each (team, points) IN scores: add points to team total
  sort totals by points descending, then name ascending
  print conflicts and ranking
END
```

7. Python Source Code

```
fixtures = [
    ("Football", "10:00", "Tigers", "Eagles"),
    ("Basketball", "10:00", "Tigers", "Sharks"),
    ("Volleyball", "11:00", "Eagles", "Sharks")
]
scores = [
    ("Tigers", 3), ("Eagles", 2), ("Sharks", 1),
    ("Tigers", 2), ("Sharks", 3), ("Eagles", 1)
]

conflicts = []
for i in range(len(fixtures)):
    for j in range(i + 1, len(fixtures)):
        a = fixtures[i]
        b = fixtures[j]
        shared = []
        if a[1] == b[1]:
            for team in (a[2], a[3]):
                if team == b[2] or team == b[3]:
                    shared.append(team)
        if shared:
            conflicts.append((a[0], b[0], a[1], shared))

totals = []
```

```
for team, points in scores:
    found = False
    for index in range(len(totals)):
        if totals[index][0] == team:
            totals[index] = (team, totals[index][1] + points)
            found = True
            break
    if not found:
        totals.append((team, points))

totals.sort(key=lambda item: (-item[1], item[0]))

print("Scheduling conflicts:")
if not conflicts:
    print("None")
for first, second, slot, teams in conflicts:
    print(first, "and", second, "at", slot, "for", ", ".join(teams))
print("Final ranking:")
for rank, (team, points) in enumerate(totals, start=1):
    print(rank, team, points)
```

8. Test Cases

Test	Input / Condition	Expected Result
Same team, same slot	Tigers in two 10:00 fixtures	Conflict detected
Different teams, same slo	No shared team	No conflict
Repeated score entry	Tigers: 3 and 2	Total 5
Ranking	Tigers 5, Sharks 4, Eagles 3	Tigers first

9. Sample Output

```
Scheduling conflicts:
Football and Basketball at 10:00 for Tigers
Final ranking:
1 Tigers 5
2 Sharks 4
3 Eagles 3
```

10. Conclusion

The program uses tuple records, nested loops, aggregation, and sorting to identify fixture conflicts and rank teams.

93. Smart Public Transport Fare Reconciliation

1. Problem Statement

Calculate expected fares for passengers travelling on different routes and passenger categories, compare them with collected fares, and flag discrepancies.

2. Objectives

• Apply route-based base fares and category discounts.

• Calculate expected revenue for each trip.

• Compare expected and collected amounts and summarize differences.

3. Assumptions

• Route fares are supplied in a dictionary-like list of tuples.

• Student passengers receive 50% discount; senior passengers receive 30%; adults pay full fare.

• Each trip record is (route, category, passengers, collected_amount). Amounts are in rupees.

4. Algorithm

1. Define base fares and category multipliers.

2. Read each trip record.

3. Find the route fare and category multiplier.

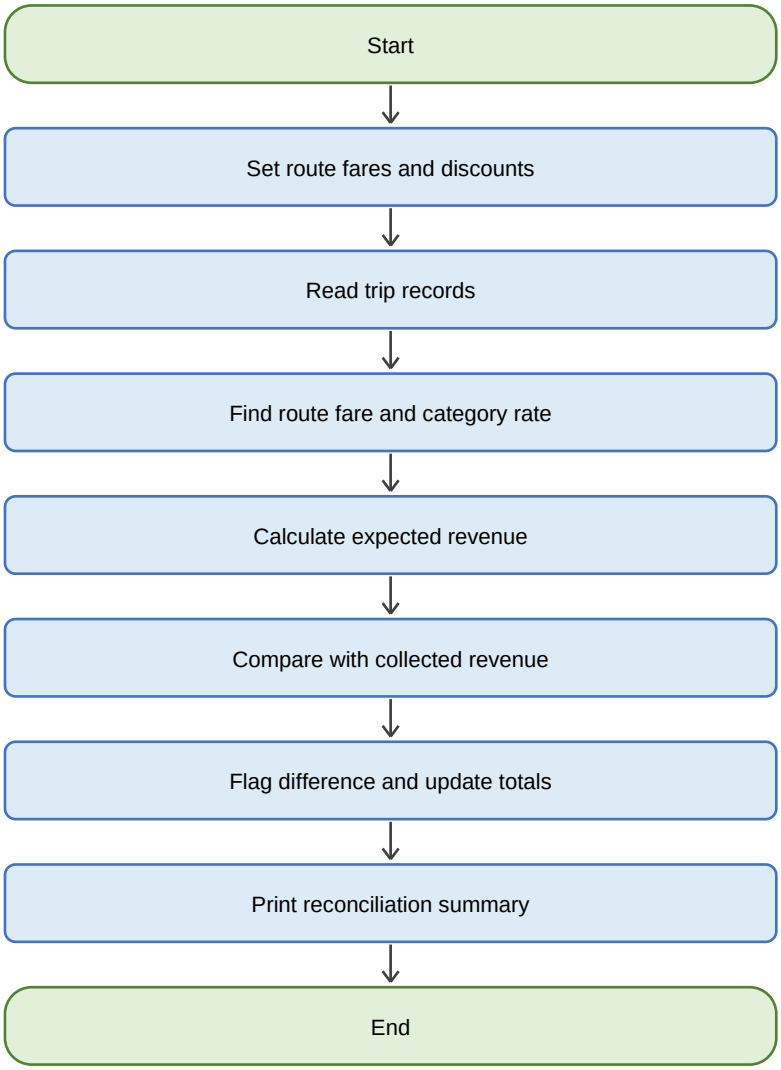
4. Calculate expected revenue = fare × multiplier × passenger count.

5. Compare expected and collected revenue and report the difference.

6. Add expected and collected amounts to totals.

5. Flowchart

Follow the arrows from top to bottom. Conditions are stated inside the relevant step boxes.



6. Pseudocode

```
BEGIN
  fares ← route fare tuples
  trips ← trip tuples
  expected_total ← 0; collected_total ← 0
  FOR each trip
    look up fare and category multiplier
    expected ← fare × multiplier × passengers
    difference ← collected - expected
    classify as MATCH or DISCREPANCY
    update totals
  print totals and net difference
END
```

7. Python Source Code

```
fares = [("R1", 20), ("R2", 30), ("R3", 40)]
rates = [("adult", 1.0), ("student", 0.5), ("senior", 0.7)]
trips = [
    ("R1", "adult", 10, 200),
    ("R2", "student", 8, 120),
    ("R3", "senior", 5, 150),
    ("R1", "student", 4, 30)
]

expected_total = 0
collected_total = 0

for route, category, passengers, collected in trips:
    fare = 0
    multiplier = 1
    for route_name, amount in fares:
        if route_name == route:
```

```
            fare = amount
        for category_name, rate in rates:
            if category_name == category:
                multiplier = rate

        expected = fare * multiplier * passengers
        difference = collected - expected
        expected_total += expected
        collected_total += collected
        status = "MATCH" if difference == 0 else "DISCREPANCY"
        print(route, category, "expected =", expected,
              "collected =", collected, "difference =", difference, status)

print("Expected total:", expected_total)
print("Collected total:", collected_total)
print("Net difference:", collected_total - expected_total)
```

8. Test Cases

Test	Input / Condition	Expected Result
Adult on R1	10 passengers; fare ■20	Expected ■200
Student on R2	8 passengers; 50% discount	Expected ■120
Senior on R3	5 passengers; 30% discount	Expected ■140
Under-collection	R1 student expected ■40, collected ■30	Difference -■10

9. Sample Output

```
R1 adult expected = 200.0 collected = 200 difference = 0.0 MATCH
R2 student expected = 120.0 collected = 120 difference = 0.0 MATCH
R3 senior expected = 140.0 collected = 150 difference = 10.0 DISCREPANCY
R1 student expected = 40.0 collected = 30 difference = -10.0 DISCREPANCY
Expected total: 500.0
Collected total: 500
Net difference: 0.0
```

10. Conclusion

The program reconciles fare revenue trip by trip and identifies both over-collection and under-collection even when the net difference is zero.

94. Disaster Relief Supply Allocation

1. Problem Statement

Compare available relief stock with area-wise demand, serve urgent areas first, allocate limited supplies, and report unmet demand and remaining stock.

2. Objectives

• Represent stock and area demand as lists of tuples.

• Prioritize areas by urgency.

• Allocate supplies without exceeding stock and calculate shortages.

3. Assumptions

• One supply item, such as water kits, is allocated in this example.

• Priority 1 is most urgent; lower priority numbers are served first.

• Areas with equal priority retain the order in which they are listed.

4. Algorithm

1. Set available stock and area demand records.

2. Sort areas by priority.

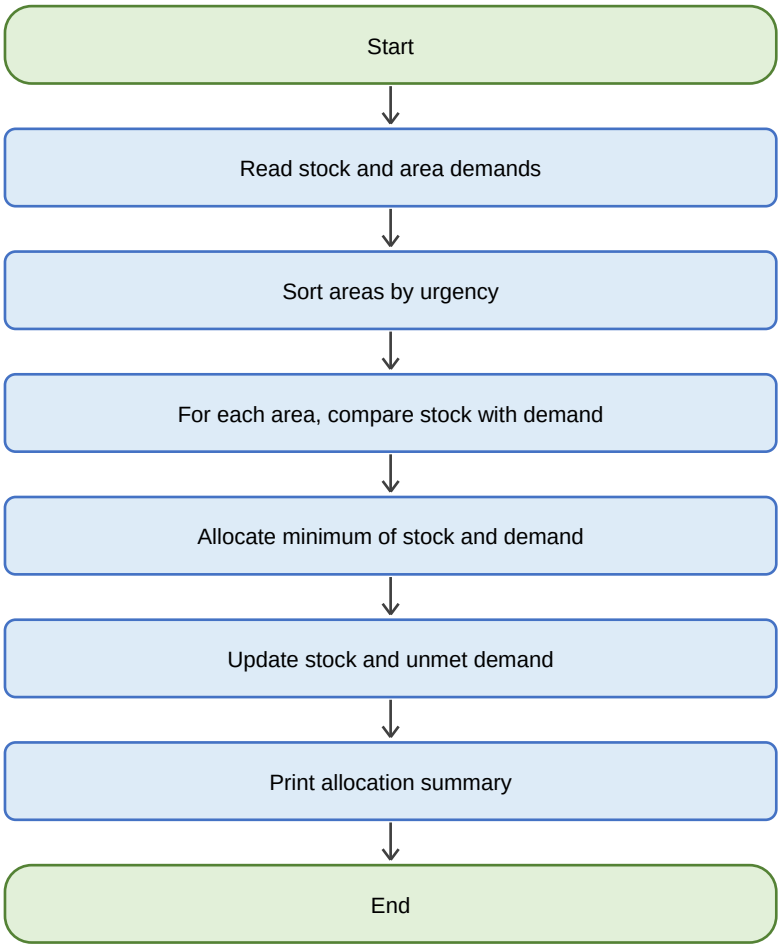
3. For each area, allocate the smaller of stock and demand.

4. Reduce stock and demand by the allocated amount.

5. Report allocations, unmet demand, and remaining stock.

5. Flowchart

Follow the arrows from top to bottom. Conditions are stated inside the relevant step boxes.



6. Pseudocode

```
BEGIN
  stock ← available quantity
  areas ← (area, priority, demand) tuples
  sort areas by priority ascending
  FOR each area
    allocated ← minimum(stock, demand)
    unmet ← demand - allocated
    stock ← stock - allocated
    print area, allocated, unmet
  print remaining stock
END
```

7. Python Source Code

```
stock = 120
areas = [
    ("North", 2, 50),
    ("Coastal", 1, 70),
    ("Hill", 3, 40),
    ("River", 1, 30)
]
areas.sort(key=lambda area: area[1])

total_unmet = 0
print("Area | Priority | Demand | Allocated | Unmet")
for name, priority, demand in areas:
    allocated = min(stock, demand)
    unmet = demand - allocated
    stock -= allocated
    total_unmet += unmet
    print(name, priority, demand, allocated, unmet)

print("Total unmet demand:", total_unmet)
print("Remaining stock:", stock)
```

8. Test Cases

Test	Input / Condition	Expected Result
Stock covers demand	Stock 100; demand 60	Allocate 60; stock 40
Urgent priority	Priority 1 and 3 areas	Priority 1 served first
Stock exhausted	Demand exceeds stock	Unmet demand reported
No demand	Demand 0	Allocation 0

9. Sample Output

```
Area | Priority | Demand | Allocated | Unmet
Coastal 1 70 70 0
River 1 30 30 0
North 2 50 20 30
Hill 3 40 0 40
Total unmet demand: 70
Remaining stock: 0
```

10. Conclusion

The program demonstrates priority-based allocation while ensuring distributed quantities never exceed available stock.

95. Integrated Manufacturing Production Planner

1. Problem Statement

Evaluate production orders against raw-material stock, estimate production feasibility, record defective units, and identify shortages and quality violations.

2. Objectives

• Calculate material required for each order.

• Check production feasibility from available materials.

• Calculate good units, defect rate, and quality status.

3. Assumptions

• Each finished unit requires 2 kg of material.

• A defect rate above 5% is a quality violation.

• The planner evaluates each order independently against the available stock; it does not reserve material for future orders.

4. Algorithm

1. Read raw material stock and production orders.

2. Calculate material required = units × material per unit.

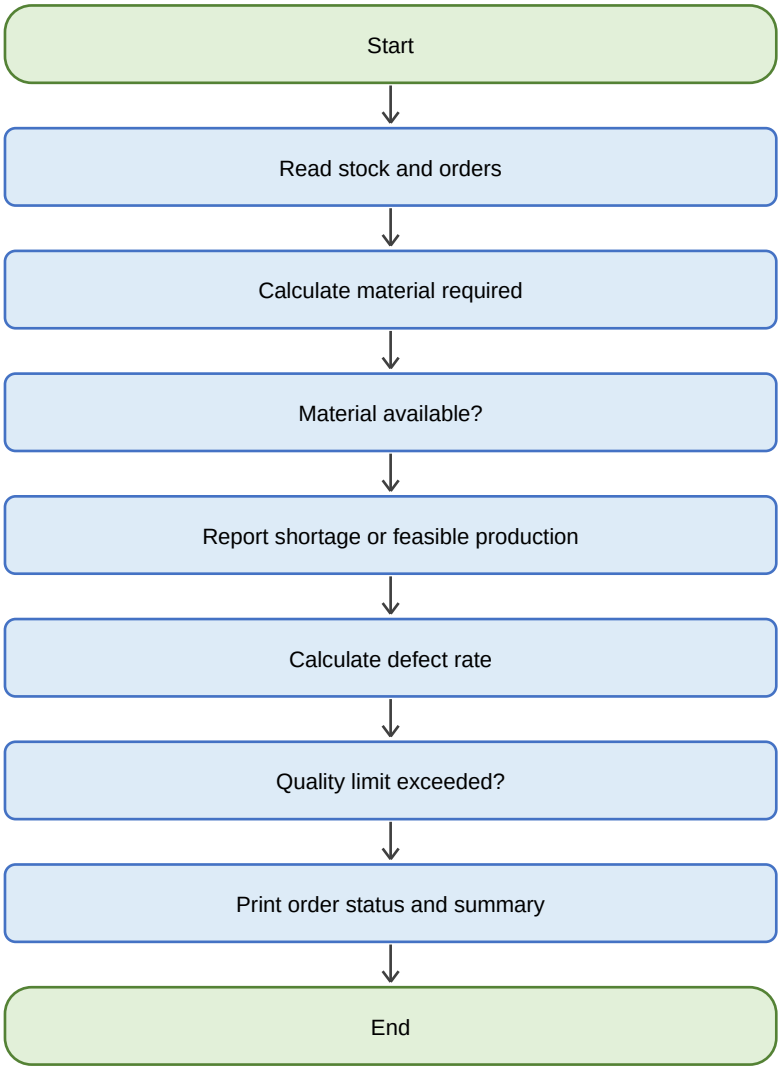
3. Compare required material with stock and report shortage if needed.

4. Calculate defect rate from produced and defective units.

5. Classify quality as PASS or VIOLATION and report good units.

5. Flowchart

Follow the arrows from top to bottom. Conditions are stated inside the relevant step boxes.



6. Pseudocode

```
BEGIN
  stock ← raw material quantity
  FOR each order (name, units, produced, defects)
    required ← units × material_per_unit
    IF required > stock: report material shortage
    ELSE report material feasible
    defect_rate ← defects / produced × 100 (if produced > 0)
    IF defect_rate > limit: report quality violation
    print good units and order summary
END
```

7. Python Source Code

```
stock_kg = 500
material_per_unit = 2
quality_limit = 5
orders = [
    ("A", 200, 190, 5),
    ("B", 180, 180, 12),
    ("C", 100, 95, 0)
]

for name, planned_units, produced, defects in orders:
    required = planned_units * material_per_unit
    shortage = max(0, required - stock_kg)
    material_status = "FEASIBLE" if shortage == 0 else "SHORTAGE"
    if produced > 0:
        defect_rate = defects / produced * 100
    else:
        defect_rate = 0
    quality_status = "VIOLATION" if defect_rate > quality_limit else "PASS"
    good_units = produced - defects
```

```
    print("Order", name, "| required kg:", required,
          "| material:", material_status, "| shortage kg:", shortage)
    print("  good units:", good_units, "| defect rate:",
          round(defect_rate, 2), "% | quality:", quality_status)
```

8. Test Cases

Test	Input / Condition	Expected Result
Material feasible	100 units × 2 kg; stock 500 kg	Required 200 kg
Material shortage	300 units × 2 kg; stock 500 kg	Shortage 100 kg
Quality pass	5 defects / 190 produced	2.63%, PASS
Quality violation	12 defects / 180 produced	6.67%, VIOLATION

9. Sample Output

```
Order A | required kg: 400 | material: FEASIBLE | shortage kg: 0
  good units: 185 | defect rate: 2.63 % | quality: PASS
Order B | required kg: 360 | material: FEASIBLE | shortage kg: 0
  good units: 168 | defect rate: 6.67 % | quality: VIOLATION
Order C | required kg: 200 | material: FEASIBLE | shortage kg: 0
  good units: 95 | defect rate: 0.0 % | quality: PASS
```

10. Conclusion

The program combines material planning with production-quality checks to show shortages, good output, and quality violations.

96. Multi-Criteria Library Resource Distribution

1. Problem Statement

Allocate available book copies across library branches according to priority and demand, then calculate unfulfilled demand and remaining copies.

2. Objectives

• Store branch demand and priority as tuples.

• Allocate limited copies using a defined priority order.

• Produce branch-level and overall unmet-demand summaries.

3. Assumptions

• This example allocates one title with 100 available copies.

• Priority 1 is highest and is processed first.

• For equal priority, the branch with higher demand is served first.

4. Algorithm

1. Read available copies and branch demand.

2. Sort branches by priority and then demand descending.

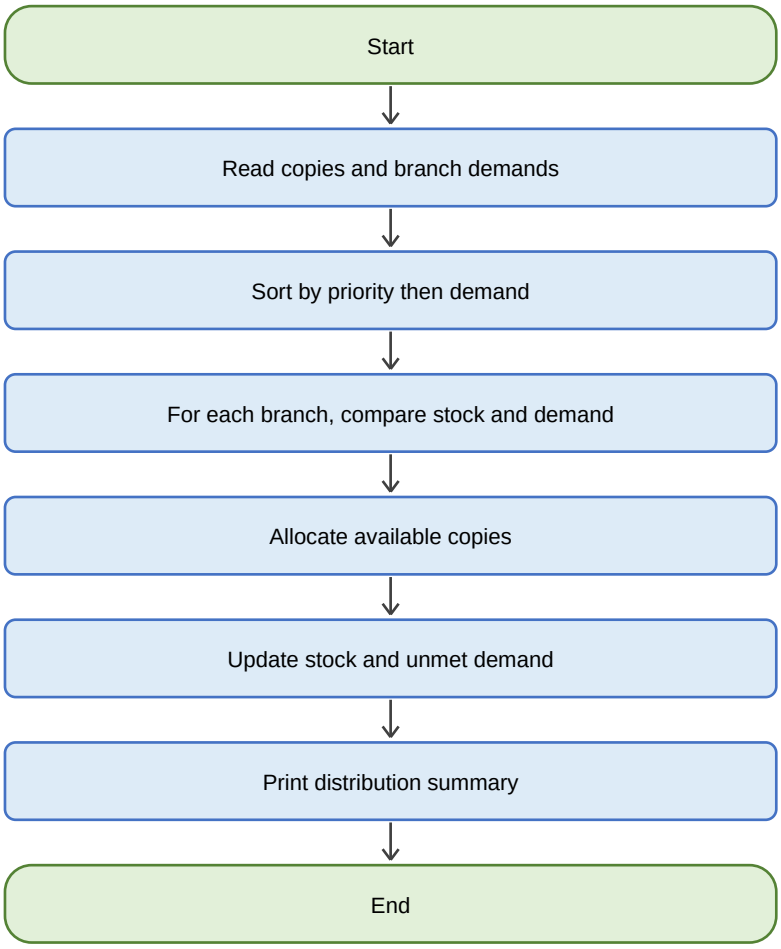
3. Allocate the minimum of available copies and requested copies.

4. Update remaining stock and unmet demand.

5. Print the allocation and summary.

5. Flowchart

Follow the arrows from top to bottom. Conditions are stated inside the relevant step boxes.



6. Pseudocode

```
BEGIN
  copies ← available copies
  branches ← (branch, priority, demand) tuples
  sort by priority ascending, demand descending
  FOR each branch
    allocated ← minimum(copies, demand)
    unmet ← demand - allocated
    copies ← copies - allocated
    print branch allocation
  print total unmet and remaining copies
END
```

7. Python Source Code

```
copies = 100
branches = [
    ("East", 2, 35),
    ("Central", 1, 50),
    ("West", 1, 40),
    ("South", 3, 30)
]
branches.sort(key=lambda item: (item[1], -item[2]))

total_unmet = 0
print("Branch | Priority | Demand | Allocated | Unmet")
for branch, priority, demand in branches:
    allocated = min(copies, demand)
    unmet = demand - allocated
    copies -= allocated
    total_unmet += unmet
    print(branch, priority, demand, allocated, unmet)

print("Unmet demand:", total_unmet)
print("Copies remaining:", copies)
```

8. Test Cases

Test	Input / Condition	Expected Result
High priority	Central priority 1, demand 50	Processed before priority 2
Tie-break	Central 50 and West 40 both priority 1	Central first
Insufficient stock	Total demand above 100	Unmet demand shown
All demand served	Stock exceeds demand	Remaining copies shown

9. Sample Output

```
Branch | Priority | Demand | Allocated | Unmet
Central 1 50 50 0
West 1 40 40 0
East 2 35 10 25
South 3 30 0 30
Unmet demand: 55
Copies remaining: 0
```

10. Conclusion

The program applies transparent priority rules to library distribution and quantifies the demand that cannot be met.

97. Smart Campus Incident and Maintenance Prioritization

1. Problem Statement

Classify hypothetical campus maintenance reports using severity, location, and frequency, rank incidents using predefined rules, and summarize priorities.

2. Objectives

• Convert report attributes into a priority score.

• Use severity, sensitive location, and frequency consistently.

• Sort incidents and count high-priority cases.

3. Assumptions

• Severity is 1–5, where 5 is most severe.

• Sensitive locations such as laboratory, electrical room, or main gate add 2 points.

• Frequency of 3 or more reports adds 1 point.

• Score 7 or more is HIGH; 5–6 is MEDIUM; below 5 is LOW.

4. Algorithm

1. Read incident tuples.

2. Start each score at severity.

3. Add location points for sensitive locations.

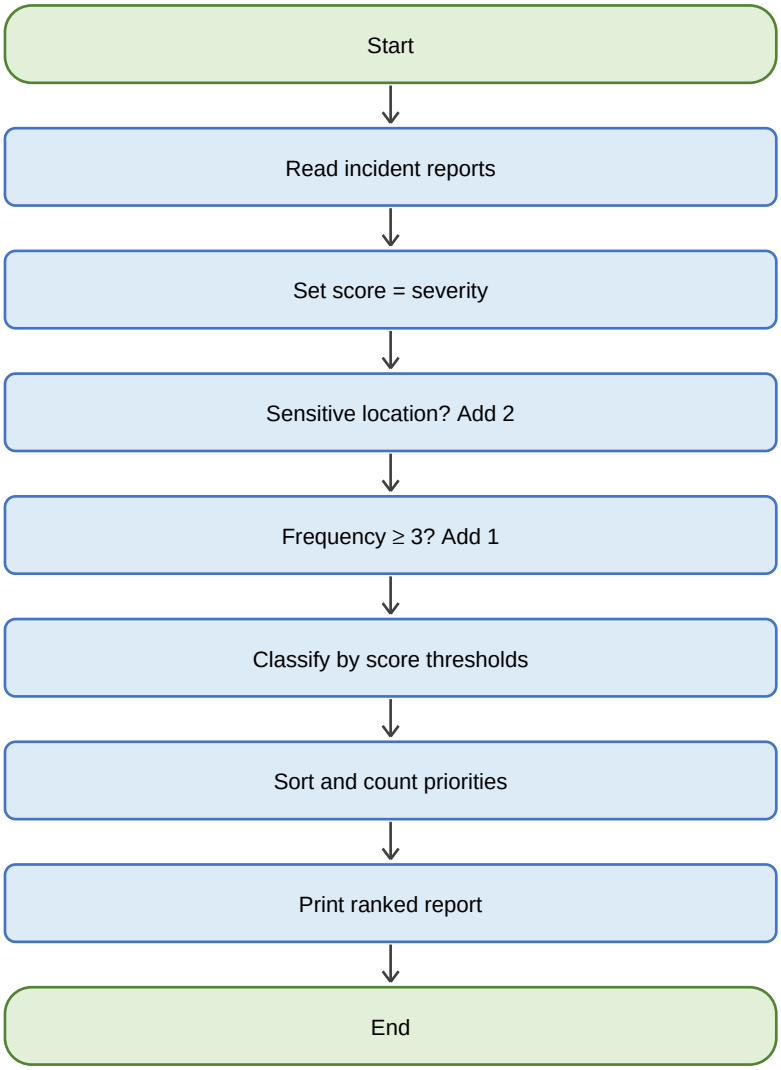
4. Add a frequency point when reports are 3 or more.

5. Assign HIGH, MEDIUM, or LOW using thresholds.

6. Sort by score descending and print the priority summary.

5. Flowchart

Follow the arrows from top to bottom. Conditions are stated inside the relevant step boxes.



6. Pseudocode

```
BEGIN
  reports ← incident tuples
  FOR each report
    score ← severity
    IF location is sensitive: score ← score + 2
    IF frequency ≥ 3: score ← score + 1
    classify score as HIGH, MEDIUM, or LOW
  sort reports by score descending
  print ranked reports and counts
END
```

7. Python Source Code

```
reports = [
    ("Loose wire", "electrical room", 5, 2),
    ("Broken chair", "classroom", 2, 1),
    ("Water leak", "laboratory", 4, 4),
    ("Blocked entrance", "main gate", 3, 3)
]
sensitive = ["laboratory", "electrical room", "main gate"]
ranked = []

for title, location, severity, frequency in reports:
    score = severity
    if location in sensitive:
        score += 2
    if frequency >= 3:
        score += 1
    if score >= 7:
        level = "HIGH"
    elif score >= 5:
        level = "MEDIUM"
```

```
    else:
        level = "LOW"
    ranked.append((score, title, location, level))

ranked.sort(key=lambda item: (-item[0], item[1]))
counts = [("HIGH", 0), ("MEDIUM", 0), ("LOW", 0)]

for score, title, location, level in ranked:
    for i in range(len(counts)):
        if counts[i][0] == level:
            counts[i] = (level, counts[i][1] + 1)
    print(title, "|", location, "| score:", score, "|", level)

print("Priority summary:", counts)
```

8. Test Cases

Test	Input / Condition	Expected Result
High severity sensitive location	Severity 5, electrical room	Score 7, HIGH
Frequent issue	Frequency 3 or more	Add 1 point
Low severity ordinary location	Severity 2, classroom, frequency 1	Score 2, LOW
Tie score	Two equal scores	Alphabetical title order

9. Sample Output

```
Loose wire | electrical room | score: 7 | HIGH
Water leak | laboratory | score: 7 | HIGH
Blocked entrance | main gate | score: 6 | MEDIUM
Broken chair | classroom | score: 2 | LOW
Priority summary: [('HIGH', 2), ('MEDIUM', 1), ('LOW', 1)]
```

10. Conclusion

The program converts incident details into a consistent score, making maintenance triage more transparent and repeatable.

98. University Academic Progression Audit

1. Problem Statement

Combine semester marks, pass conditions, attendance, and prerequisite completion to decide whether a student may progress and identify outstanding requirements.

2. Objectives

• Check subject-wise pass marks.

• Validate attendance against a minimum threshold.

• Check prerequisite completion and summarize eligibility.

3. Assumptions

• Each subject record is (subject, marks, pass_mark).

• Minimum attendance is 75%.

• Progression requires every listed subject to pass, attendance to meet the threshold, and all prerequisites to be complete.

4. Algorithm

1. Read subject records, attendance, and prerequisite status.

2. Compare each subject mark with its pass mark.

3. Collect failed subjects.

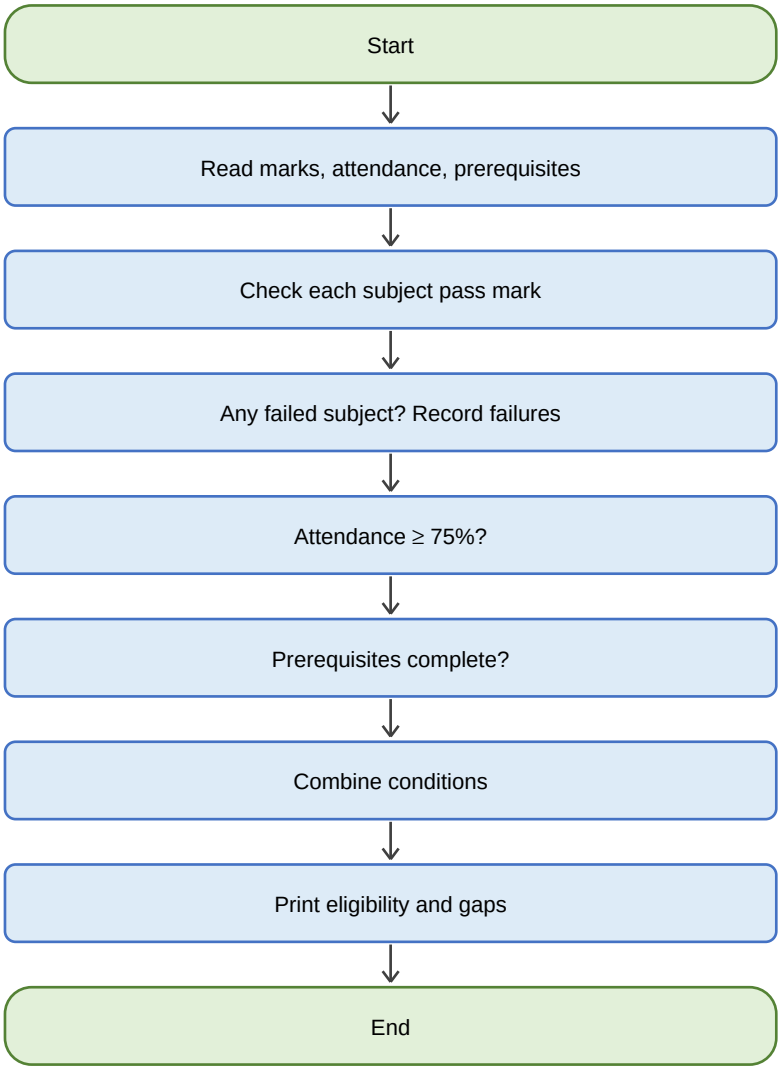
4. Check attendance and prerequisite completion.

5. If all conditions pass, mark ELIGIBLE; otherwise NOT ELIGIBLE.

6. Print the decision and outstanding requirements.

5. Flowchart

Follow the arrows from top to bottom. Conditions are stated inside the relevant step boxes.



6. Pseudocode

```
BEGIN
  failed_subjects ← empty list
  FOR each (subject, marks, pass_mark)
    IF marks < pass_mark: add subject to failed_subjects
  check attendance ≥ 75%
  check prerequisites complete
  IF no failures AND attendance valid AND prerequisites complete
    status ← ELIGIBLE
  ELSE status ← NOT ELIGIBLE
  print status and every outstanding requirement
END
```

7. Python Source Code

```
subjects = [
    ("Programming", 68, 40),
    ("Mathematics", 35, 40),
    ("English", 55, 35)
]
attendance = 78
prerequisites_complete = False
minimum_attendance = 75

failed = []
for subject, marks, pass_mark in subjects:
    if marks < pass_mark:
        failed.append(subject)

issues = []
if failed:
    issues.append("Failed subjects: " + ", ".join(failed))
if attendance < minimum_attendance:
```

```
    issues.append("Attendance below 75%")
if not prerequisites_complete:
    issues.append("Prerequisites incomplete")

if len(issues) == 0:
    status = "ELIGIBLE"
else:
    status = "NOT ELIGIBLE"

print("Progression status:", status)
if issues:
    print("Outstanding requirements:")
    for issue in issues:
        print("-", issue)
else:
    print("All requirements satisfied.")
```

8. Test Cases

Test	Input / Condition	Expected Result
All conditions pass	All marks pass, attendance 80%, prerequisites complete	ELIGIBLE
Subject fail	One mark below pass mark	NOT ELIGIBLE
Attendance fail	Attendance 70%	NOT ELIGIBLE
Prerequisite missing	Prerequisites incomplete	NOT ELIGIBLE

9. Sample Output

```
Progression status: NOT ELIGIBLE
Outstanding requirements:
- Failed subjects: Mathematics
- Prerequisites incomplete
```

10. Conclusion

The audit combines academic, attendance, and prerequisite rules and clearly lists the exact conditions preventing progression.

99. Integrated E-Commerce Order Processing and Reconciliation

1. Problem Statement

Process orders with discounts, dispatch quantities, returns, and stock changes; calculate revenue, identify discrepancies, and produce a final inventory report.

2. Objectives

• Calculate discounted order values.

• Update stock for dispatches and returns.

• Compare expected revenue with recorded collections and report mismatches.

3. Assumptions

• Product records are (name, unit_price, opening_stock).

• Order records are (product, ordered_qty, discount_percent, dispatched_qty, returned_qty, collected_amount).

• Revenue is based on net dispatched quantity after returns, using the order's discount rate.

4. Algorithm

1. Load product stock and order records.

2. Find unit price and opening stock for each product.

3. Calculate net quantity = dispatched minus returned.

4. Calculate expected revenue using discounted price and net quantity.

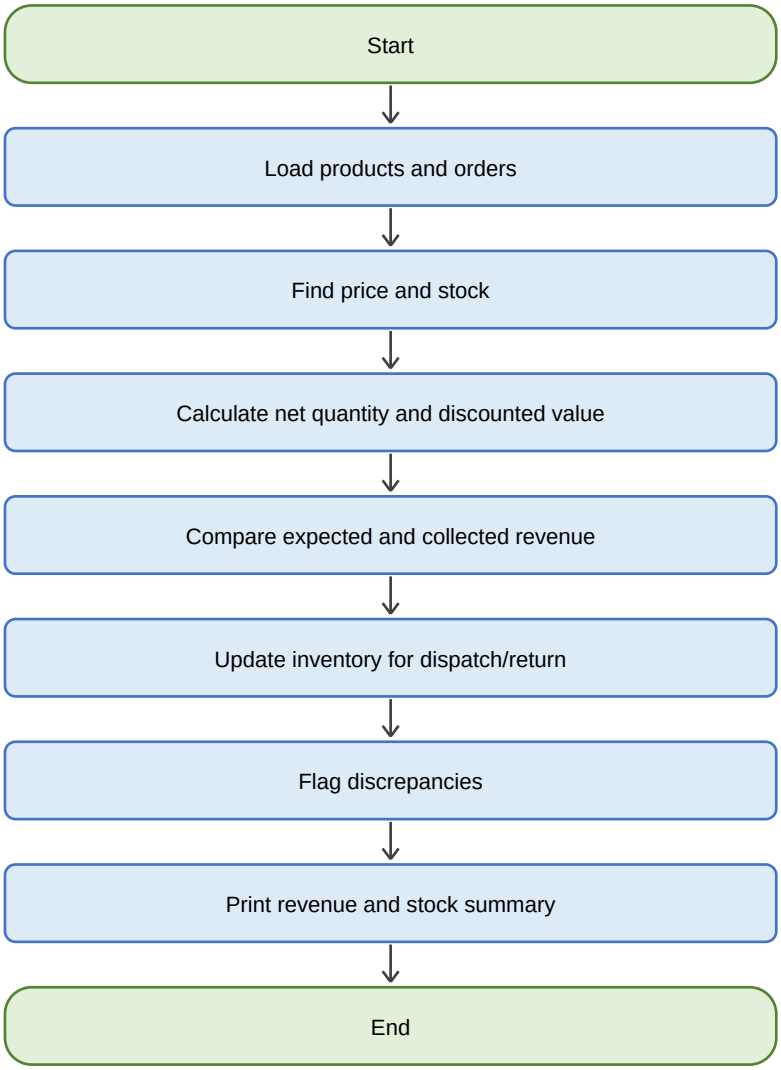
5. Compare collected revenue with expected revenue.

6. Update stock as opening stock minus dispatched plus returned.

7. Print reconciliation and final inventory.

5. Flowchart

Follow the arrows from top to bottom. Conditions are stated inside the relevant step boxes.



6. Pseudocode

```
BEGIN
  products ← product tuples
  orders ← order tuples
  FOR each order
    look up price and stock
    net_qty ← dispatched - returned
    expected ← price × (1 - discount/100) × net_qty
    difference ← collected - expected
    stock ← stock - dispatched + returned
    report order and discrepancy
  print total expected, collected, and final stock
END
```

7. Python Source Code

```
products = [
    ["Notebook", 50, 100],
    ["Pen", 10, 200]
]
orders = [
    ("Notebook", 10, 10, 8, 1, 315),
    ("Pen", 20, 0, 20, 2, 180)
]

expected_total = 0
collected_total = 0

for product_name, ordered, discount, dispatched, returned, collected in orders:
    product = None
    for item in products:
        if item[0] == product_name:
            product = item
```

```
            break

    if product is None:
        print("Unknown product:", product_name)
        continue

    price = product[1]
    net_qty = dispatched - returned
    expected = price * (1 - discount / 100) * net_qty
    difference = collected - expected
    product[2] = product[2] - dispatched + returned
    expected_total += expected
    collected_total += collected

    status = "MATCH" if difference == 0 else "DISCREPANCY"
    print(product_name, "| expected:", round(expected, 2),
          "| collected:", collected, "| difference:", round(difference, 2), status)

print("Expected revenue:", round(expected_total, 2))
print("Collected revenue:", round(collected_total, 2))
print("Final inventory:")
for name, price, stock in products:
    print(name, "stock =", stock)
```

8. Test Cases

Test	Input / Condition	Expected Result
Discounted order	10 notebooks; 10% discount; net 7	Expected ■315
Return update	20 pens dispatched, 2 returned	Stock decreases by 18
Revenue mismatch	Collected amount differs from expected	DISCREPANCY
Unknown product	Order product not in product list	Report unknown product

9. Sample Output

```
Notebook | expected: 315.0 | collected: 315 | difference: 0.0 MATCH
Pen | expected: 180.0 | collected: 180 | difference: 0.0 MATCH
Expected revenue: 495.0
Collected revenue: 495.0
Final inventory:
Notebook stock = 93
Pen stock = 182
```

10. Conclusion

The program links order values, returns, revenue reconciliation, and stock updates in one workflow.

100. Comprehensive Smart Campus Management Simulation

1. Problem Statement

Develop a menu-driven campus program that manages student records, attendance, marks, event registrations, resource allocation, and summary reports using the prescribed Python fundamentals.

2. Objectives

• Provide a simple menu to run campus-management tasks.

• Store student data in lists and nested lists, and use tuples for event/resource records.

• Validate inputs, calculate summaries, and repeat until the user exits.

3. Assumptions

• This is a small in-memory simulation; data resets when the program ends.

• Student record format is [ID, name, attendance_percent, marks].

• Event registrations and resource allocations are stored as lists of tuples.

4. Algorithm

1. Initialize student, event, and resource lists.

2. Display a menu repeatedly.

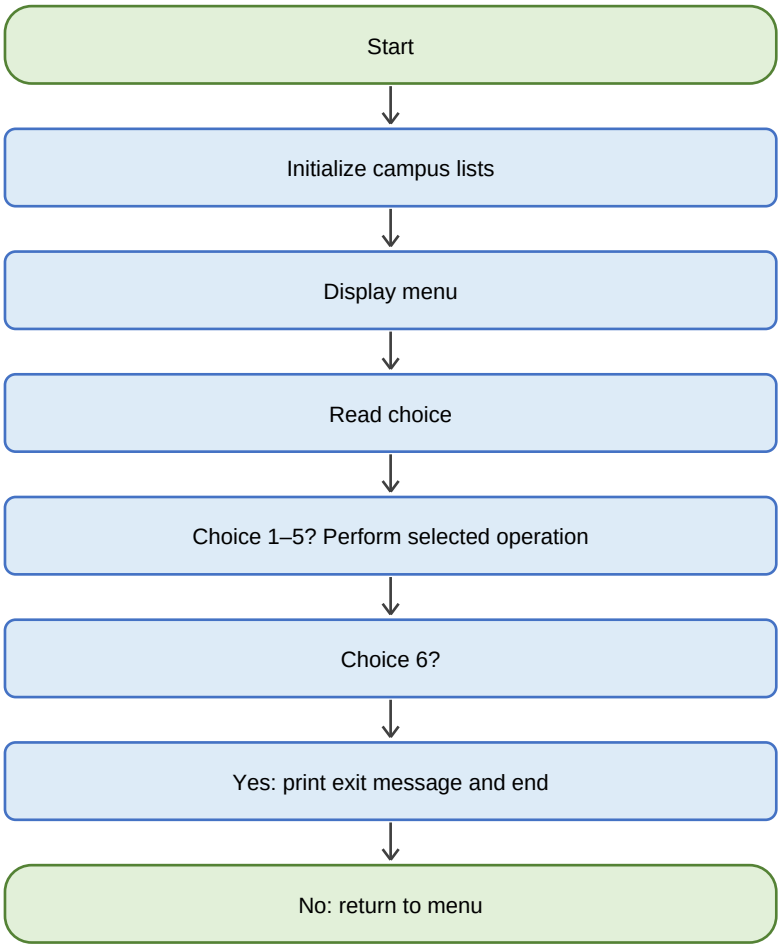
3. For each menu choice, add/list students, register for events, allocate resources, or print a summary.

4. Validate IDs and numeric values where relevant.

5. Exit only when the user chooses option 6.

5. Flowchart

Follow the arrows from top to bottom. Conditions are stated inside the relevant step boxes.



6. Pseudocode

```
BEGIN
  students ← empty list; registrations ← empty list; resources ← empty list
  REPEAT
    display menu: add student, list students, register event, allocate resource, summary, exit
    read choice
    IF choice = 1: read and append student record
    ELSE IF choice = 2: display student records
    ELSE IF choice = 3: append event registration tuple
    ELSE IF choice = 4: append resource allocation tuple
    ELSE IF choice = 5: display counts and average marks
    ELSE IF choice = 6: exit
    ELSE: display invalid choice
  UNTIL choice = 6
END
```

7. Python Source Code

```
students = []          # [ID, name, attendance, marks]
registrations = []     # (student_id, event_name)
resources = []         # (resource_name, student_id, quantity)

while True:
    print("\nSMART CAMPUS MENU")
    print("1. Add student")
    print("2. List students")
    print("3. Register for event")
    print("4. Allocate resource")
    print("5. Summary report")
    print("6. Exit")
    choice = input("Enter choice: ")

    if choice == "1":
        student_id = input("Student ID: ").strip()
        exists = False
        for student in students:
            if student[0] == student_id:
                exists = True
```

```
        if exists or student_id == "":
            print("Invalid or duplicate ID.")
        else:
            name = input("Name: ").strip()
            attendance = float(input("Attendance percentage: "))
            marks = float(input("Marks out of 100: "))
            if 0 <= attendance <= 100 and 0 <= marks <= 100:
                students.append([student_id, name, attendance, marks])
                print("Student added.")
            else:
                print("Attendance and marks must be 0–100.")

    elif choice == "2":
        if not students:
            print("No student records.")
        for student_id, name, attendance, marks in students:
            print(student_id, name, attendance, marks)

    elif choice == "3":
        student_id = input("Student ID: ").strip()
        event_name = input("Event name: ").strip()
        found = False
        for student in students:
            if student[0] == student_id:
                found = True
        if found and event_name:
            registrations.append((student_id, event_name))
            print("Event registration saved.")
        else:
            print("Student not found or event name is empty.")

    elif choice == "4":
        resource_name = input("Resource name: ").strip()
        student_id = input("Student ID: ").strip()
        quantity = int(input("Quantity: "))
        if quantity > 0 and resource_name:
            resources.append((resource_name, student_id, quantity))
            print("Allocation recorded.")
        else:
            print("Enter a resource and positive quantity.")

    elif choice == "5":
        total_marks = 0
        for student in students:
            total_marks += student[3]
        average = total_marks / len(students) if students else 0
        print("Students:", len(students))
        print("Event registrations:", len(registrations))
        print("Resource allocations:", len(resources))
        print("Average marks:", round(average, 2))

    elif choice == "6":
        print("Exiting Smart Campus Management.")
        break

    else:
        print("Invalid choice. Select 1 to 6.")
```

8. Test Cases

Test	Input / Condition	Expected Result
Add student	Unique ID and valid values	Student added
Duplicate ID	Add same ID twice	Duplicate ID rejected
Invalid marks	Marks = 120	Range error shown
Register event	Existing student and event name	Registration saved
Exit	Choice 6	Program ends

9. Sample Output

```
SMART CAMPUS MENU
1. Add student
2. List students
3. Register for event
4. Allocate resource
5. Summary report
6. Exit
Enter choice: 1
```

```
Student ID: S101
Name: Asha
Attendance percentage: 88
Marks out of 100: 91
Student added.

Enter choice: 5
Students: 1
Event registrations: 0
Resource allocations: 0
Average marks: 91.0
```

10. Conclusion

The menu-driven simulation combines core syllabus concepts into a practical campus workflow. It uses in-memory lists and tuples and can be extended with file storage and stronger validation.